

Skill -3&4(26-08-26)

3.To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10

int main() {
    char *history[MAX_HISTORY];
    int count = 0;
    char buffer[200];

    while (1) {
        printf("myshell> ");
        fgets(buffer, sizeof(buffer), stdin);

        buffer[strcspn(buffer, "\n")] = '\0';

        if (strcmp(buffer, "exit") == 0)
            break;

        if (strcmp(buffer, "history") == 0) {
            for (int i = 0; i < count; i++)
                printf("%d  %s\n", i + 1, history[i]);
            continue;
        }

        if (strlen(buffer) > 0) {
            if (count == MAX_HISTORY) {
                free(history[0]);

                for (int i = 1; i < MAX_HISTORY; i++)
                    history[i - 1] = history[i];

                count--;
            }

            history[count] = malloc(strlen(buffer) + 1);
            strcpy(history[count], buffer);
            count++;
        }
    }
}
```

```
manisha@LAPTOP-5SMR05RJ:~/OSlab$ nano simple_shell.c
manisha@LAPTOP-5SMR05RJ:~/OSlab$ nano shell_with_history.c
manisha@LAPTOP-5SMR05RJ:~/OSlab$ gcc shell_with_history.c -o shell_with_history
manisha@LAPTOP-5SMR05RJ:~/OSlab$ ./shell_with_history
-bash: ./: Is a directory
```

```

        history[count] = malloc(strlen(buffer) + 1);
        strcpy(history[count], buffer);
        count++;
    }
}

for (int i = 0; i < count; i++)
    free(history[i]);

return 0;
}

```

4.To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```

manisha@LAPTOP-5SMR05RJ:~/OSlab$ gcc command_tokenizer.c -o command_tokenizer
manisha@LAPTOP-5SMR05RJ:~/OSlab$ ./command_tokenizer
Enter command: gcc command_tokenizer.c -o command_tokenizer
Tokens:
gcc
command_tokenizer.c
-o
command_tokenizer

```

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char *token;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    token = strtok(input, " \t");

    printf("Tokens:\n");

    while (token != NULL) {
        printf("%s\n", token);
        token = strtok(NULL, " \t");
    }

    return 0;
}
```